

Guía de Arquitectura del Backend — Kosmos

Para programadores que vienen del mundo MVC y quieren entender qué está pasando aquí.

Índice

1. [¿Qué es esto y por qué no parece MVC?](#)
 2. [El stack tecnológico](#)
 3. [La arquitectura en una imagen](#)
 4. [Las tres capas explicadas](#)
 - o [Dominio](#)
 - o [Aplicación](#)
 - o [Infraestructura](#)
 5. [¿Qué son los Puertos?](#)
 6. [Inyección de dependencias con TSyringe](#)
 7. [Flujo completo de una petición](#)
 8. [Módulos del proyecto](#)
 9. [La base de datos \(Prisma + PostgreSQL\)](#)
 10. [Autenticación JWT](#)
 11. [Manejo de errores](#)
 12. [Resumen: MVC vs Clean Architecture](#)
-

1. ¿Qué es esto y por qué no parece MVC?

Si vienes de MVC, estás acostumbrado a esta estructura:

```
Model → Controller → View
```

En MVC, el **Controller** recibe la petición, le habla directamente al **Model** (que toca la base de datos), y devuelve una **View** (HTML o JSON).

El backend de Kosmos usa una arquitectura diferente llamada **Clean Architecture** (también conocida como Arquitectura Hexagonal o Puertos y Adaptadores). La razón es una idea muy simple pero poderosa:

"El núcleo de tu aplicación no debe saber nada del mundo exterior"

¿Qué significa "el mundo exterior"? La base de datos, Express, la librería de JWT, bcrypt... todo eso es un detalle. La lógica del negocio (las reglas de qué puede y no puede hacer el usuario) vive en el centro y no depende de ninguna librería externa.

¿Por qué molestarse?

- Puedes cambiar PostgreSQL por MongoDB sin tocar la lógica de negocio.
- Puedes testear la lógica sin levantar un servidor Express ni una base de datos.
- Cada pieza tiene una responsabilidad clara y delimitada.

2. El stack tecnológico

Qué hace	Tecnología
Framework HTTP	Express.js
Lenguaje	TypeScript
Base de datos	PostgreSQL
ORM	Prisma
Inyección de dependencias	TSyringe
Validación de entrada	Zod
Autenticación	JWT (jsonwebtoken) + bcrypt
Notificaciones push	web-push (VAPID)
Monorepo	pnpm workspaces + Turborepo

3. La arquitectura en una imagen



La clave: **las flechas de dependencia siempre apuntan hacia adentro**. El dominio no sabe que existe Prisma ni Express.

4. Las tres capas explicadas

4.1 Capa de Dominio

Ruta: `src/modules/*/domain/`

Esta es la capa más importante. Contiene las **entidades** y las **reglas de negocio**. No importa ninguna librería externa (ni Prisma, ni Express, ni nada).

Entidades

Una entidad es un objeto que tiene **identidad** (un ID) y **reglas propias**. Ejemplo: `Task`:

```
// src/modules/tasks/domain/task.entity.ts

export class Task {
  private constructor(private props: TaskProps) {}

  // La única forma de crear una Task válida
  static create(props: TaskProps): Task {
    if (props.title.trim().length === 0) {
      throw new ValidationError("El título de la tarea no puede estar vacío");
    }
    return new Task(props);
  }

  // Para cambiar el estado, hay que usar este método
  changeStatus(status: TaskStatus): void {
    this.props.status = status;
    this.props.updatedAt = new Date();
  }
}
```

Diferencia con MVC: En MVC el Model suele ser solo una representación de la tabla de la base de datos (un DTO glorificado). Aquí la entidad **contiene lógica**: no puedes crear una tarea con título vacío, y para cambiar su estado debes llamar a un método específico.

El constructor es `private`, lo que obliga a usar `Task.create()`. Esto garantiza que **nunca puede existir un Task inválido en memoria**.

Value Objects

Son objetos que no tienen ID, su identidad es su valor. Ejemplo: `SpaceViewType`:

```
// src/modules/spaces/domain/space-view-type.vo.ts
export type SpaceViewType = "LIST" | "CALENDAR" | "KANBAN";
```

`TaskStatus` es otro Value Object: `"TODO" | "IN_PROGRESS" | "DONE"`. No son entidades porque no tienen identidad propia; son simplemente valores con reglas.

Los Puertos (interfaces del dominio)

Dentro de `domain/ports.ts` hay **interfaces** que describen cómo el dominio quiere comunicarse con el exterior. Ejemplo del módulo `auth` :

```
// src/modules/auth/domain/ports.ts

// Puerto de SALIDA: "necesito algo que sepa buscar usuarios"
export interface UserRepository {
  findById(id: string): Promise<User | null>;
  findByEmail(email: string): Promise<User | null>;
  save(user: User): Promise<void>;
}

// Puerto de SALIDA: "necesito algo que sepa hashear contraseñas"
export interface PasswordHasher {
  hash(plainText: string): Promise<string>;
  compare(plainText: string, hash: string): Promise<boolean>;
}

// Puerto de ENTRADA: "alguien de afuera puede pedirme hacer login"
export interface LoginUserUseCase {
  execute(input: LoginUserInput): Promise<AuthResult>;
}
```

4.2 Capa de Aplicación

Ruta: `src/modules/*/application/use-cases/`

Los **Use Cases** (Casos de Uso) son los directores de orquesta. Cada caso de uso representa **una acción que el usuario puede hacer** en el sistema.

Ejemplos de use cases reales del proyecto:

- `CreateTaskUseCase` — crear una tarea
- `LoginUserUseCase` — iniciar sesión
- `ToggleSpaceUseCase` — activar/desactivar un espacio
- `ReorderTasksUseCase` — reordenar tareas con drag & drop

Veamos `CreateTaskUseCase` completo:

```
// src/modules/tasks/application/use-cases/create-task.use-case.ts

@injectable()
export class CreateTaskUseCase implements CreateTaskUseCasePort {
  constructor(
    // Recibe interfaces, no implementaciones concretas
    @inject(DI_TOKENS.TaskRepository) private readonly taskRepository: TaskRepository,
    @inject(DI_TOKENS.SpaceRepository) private readonly spaceRepository: SpaceRepository,
  ) {}

  async execute(input: CreateTaskInput): Promise<TaskProps> {
    // 1. Verificar que el usuario es dueño del espacio
    await assertSpaceOwnership(this.spaceRepository, input.spaceId, input.requesterId);

    // 2. Obtener las tareas existentes para calcular la posición
    const existingTasks = await this.taskRepository.findAllBySpace(input.spaceId);

    // 3. Crear la entidad Task (aquí se validan las reglas de negocio)
    const task = Task.create({
      id: randomUUID(),
      title: input.title,
      description: input.description ?? null,
      subtasks: [],
      status: "TODO",
      priority: input.priority ?? "MEDIUM",
      dueDate: input.dueDate ?? null,
      position: existingTasks.length,
      spaceId: input.spaceId,
      createdAt: new Date(),
      updatedAt: new Date(),
    });

    // 4. Guardar (sin saber si es Prisma, MongoDB, o un array en memoria)
    await this.taskRepository.save(task);

    // 5. Devolver un snapshot plano (no la entidad completa)
    return task.toSnapshot();
  }
}
```

Lo que hace un Use Case:

1. Valida autorización (¿tiene permiso este usuario?)
2. Carga datos necesarios
3. Crea/modifica entidades del dominio
4. Persiste los cambios
5. Devuelve el resultado

Lo que NO hace un Use Case:

- No sabe nada de HTTP (sin **req** , sin **res**)
- No sabe nada de Prisma ni de SQL
- No sabe si está corriendo en un test o en producción

4.3 Capa de Infraestructura

Ruta: `src/modules/*/infrastructure/`

Aquí viven todos los detalles técnicos. Se divide en dos subcarpetas:

`infrastructure/http/` — La parte que habla con Express

- **Router** (`auth.router.ts`): define las rutas y conecta con el Controller.
- **Controller** (`auth.controller.ts`): recibe el `Request`, valida el body con Zod, llama al Use Case, devuelve el `Response`.
- **Schemas** (`auth.schemas.ts`): los esquemas Zod para validar el JSON de entrada.

```
// src/modules/auth/infrastructure/http/auth.controller.ts

@injectable()
export class AuthController {
  constructor(
    @inject(DI_TOKENS.LoginUserUseCase) private readonly loginUserUseCase: LoginUserUseCase,
  ) {}

  login = async (req: Request, res: Response): Promise<void> => {
    // 1. Validar el body con Zod (lanza error si falla)
    const dto = loginSchema.parse(req.body);
    // 2. Llamar al Use Case
    const result = await this.loginUserUseCase.execute(dto);
    // 3. Responder
    res.status(200).json(result);
  };
}
```

```
// src/modules/auth/infrastructure/http/auth.router.ts

export function createAuthRouter(): Router {
  const router = Router();
  const controller = container.resolve(AuthController); // ← DI container

  router.post("/register", asyncHandler(controller.register));
  router.post("/login", asyncHandler(controller.login));
  router.post("/refresh", asyncHandler(controller.refresh));

  return router;
}
```

Comparación con MVC: En MVC el Controller suele hacer todo: validar, llamar al modelo, decidir la lógica. Aquí el Controller es deliberadamente tonto: solo valida la entrada y delega al Use Case.

`infrastructure/persistence/` — La parte que habla con la base de datos

Aquí están los **repositorios**: implementaciones concretas de los puertos del dominio.

```
// src/modules/auth/infrastructure/persistence/prisma-user.repository.ts

@injectable()
export class PrismaUserRepository implements UserRepository { // ← implementa el Puerto
  async findByEmail(email: string): Promise<User | null> {
    // Habla con Prisma (detalle técnico)
    const record = await prisma.user.findUnique({ where: { email } });
    // Convierte el registro de DB a una entidad del dominio
    return record ? this.toDomain(record) : null;
  }

  private toDomain(record: { id: string; email: string; ... }): User {
    return User.create(record); // Crea la entidad del dominio
  }
}
```

El método `toDomain` es clave: convierte el objeto plano que devuelve Prisma en una entidad real del dominio con todas sus reglas.

5. ¿Qué son los Puertos?

Si en MVC estás acostumbrado a que el Controller llame directamente a `UserModel.findOne(...)`, en Clean Architecture el Use Case no llama directamente a Prisma. En cambio:

1. El **dominio define una interfaz** (el Puerto):

```
interface UserRepository {
  findByEmail(email: string): Promise<User | null>;
}
```

2. La **infraestructura implementa esa interfaz** con Prisma:

```
class PrismaUserRepository implements UserRepository {
  async findByEmail(email: string): Promise<User | null> {
    return prisma.user.findUnique({ where: { email } });
  }
}
```

3. El **Use Case solo conoce la interfaz**, nunca la implementación:

```
class LoginUserUseCase {
  constructor(private userRepository: UserRepository) {} // ← interfaz
}
```

La metáfora: imagina que tienes un enchufe (el Puerto) y un adaptador (el Repositorio). Tu teléfono (el Use Case) solo necesita el enchufe estándar; no le importa si la corriente viene de la red eléctrica, un generador o una batería.

6. Inyección de dependencias con TSyringe

"¿Quién conecta entonces el Use Case con el Repositorio real?" La respuesta es el **contenedor de inyección de dependencias** (TSyringe).

Cada módulo tiene un archivo `*.module.ts` que registra qué implementación usar:

```
// src/modules/auth/auth.module.ts

export function registerAuthModule(): void {
  // "Cuando alguien pida UserRepository, dale PrismaUserRepository"
  container.registerSingleton(DI_TOKENS.UserRepository, PrismaUserRepository);
  // "Cuando alguien pida PasswordHasher, dale BcryptPasswordHasher"
  container.registerSingleton(DI_TOKENS.PasswordHasher, BcryptPasswordHasher);
  // "Cuando alguien pida LoginUserUseCase, dale esta clase"
  container.registerSingleton(DI_TOKENS.LoginUserUseCase, LoginUserUseCase);
}
```

Los tokens (`DI_TOKENS`) son simplemente strings únicos que actúan como nombres:

```
// src/shared/infrastructure/di/tokens.ts
export const DI_TOKENS = {
  UserRepository: "UserRepository",
  LoginUserUseCase: "LoginUserUseCase",
  // ...
} as const;
```

Cuando Express recibe una petición y el Router llama a `container.resolve(AuthController)`, el contenedor construye automáticamente toda la cadena:

```
AuthController
├─ LoginUserUseCase (porque está registrado en el DI)
│   ├─ PrismaUserRepository (implementa UserRepository)
│   ├─ BcryptPasswordHasher (implementa PasswordHasher)
│   └─ JwtTokenService (implementa TokenService)
```

Para un desarrollador MVC: es como un Laravel Container o Spring Boot con `@Autowired`, pero en Node.js.

Los decoradores `@injectable()` e `@inject()` son los que hacen que TSyringe sepa cómo construir cada clase:

```
@injectable() // "esta clase puede ser inyectada"
export class LoginUserUseCase {
  constructor(
    @inject(DI_TOKENS.UserRepository) // "dame lo que esté registrado como UserRepository"
    private readonly userRepository: UserRepository,
  ) {}
}
```

7. Flujo completo de una petición

Veamos qué pasa cuando el frontend hace `POST /api/auth/login` con `{ email, password }`:

```
1. Express recibe la petición HTTP
  └─ Middleware: helmet() agrega headers de seguridad
  └─ Middleware: cors() verifica el origen
  └─ Middleware: express.json() parsea el body

2. Router: auth.router.ts
  └─ Ruta POST /login → asyncHandler(controller.login)
  └─ asyncHandler envuelve la función para capturar errores async

3. Controller: auth.controller.ts → método login()
  └─ loginSchema.parse(req.body)
    └─ Si falla → ZodError → capturado por errorHandler → HTTP 400
    └─ Si pasa → dto = { email: "...", password: "..." }
  └─ loginUserUseCase.execute(dto)

4. Use Case: login-user.use-case.ts → método execute()
  └─ userRepository.findByEmail(dto.email)
    └─ Si no existe → throw UnauthorizedError("Credenciales inválidas")
  └─ passwordHasher.compare(dto.password, user.passwordHash)
    └─ Si no coincide → throw UnauthorizedError("Credenciales inválidas")
  └─ tokenService.generateTokenPair(user.id)
  └─ return { user: user.toPublic(), tokens }

5. Implementaciones reales (infraestructura):
  └─ userRepository → PrismaUserRepository.findByEmail()
    └─ prisma.user.findUnique({ where: { email } })
    └─ Convierte el registro DB → entidad User
  └─ passwordHasher → BcryptPasswordHasher.compare()
    └─ bcrypt.compare(plainText, hash)
  └─ tokenService → JwtTokenService.generateTokenPair()
    └─ jwt.sign({ sub: userId }, secret, { expiresIn: "15m" })

6. De vuelta en el Controller:
  └─ res.status(200).json(result)
  └─ El cliente recibe: { user: {...}, tokens: { accessToken, refreshToken } }
```

Si en cualquier paso se lanza un error del dominio (`UnauthorizedError`, `NotFoundError`, etc.), el middleware `errorHandler` lo captura y devuelve la respuesta HTTP apropiada.

8. Módulos del proyecto

El backend tiene 4 módulos, cada uno con la misma estructura interna:

```
src/modules/
  auth/          ← Registro, login, refresh de token
  spaces/        ← Espacios de trabajo del usuario
  tasks/         ← Tareas y subtareas dentro de un espacio
  notifications/ ← Suscripciones a notificaciones push
```

Cada módulo es autocontenido:

```
modules/tasks/
  domain/
    task.entity.ts      ← Entidad Task con sus reglas
    subtask.entity.ts   ← Entidad Subtask
    task-status.vo.ts   ← Value Object: TODO | IN_PROGRESS | DONE
    ports.ts            ← Interfaces: TaskRepository, CreateTaskUseCase ...
  application/
    use-cases/
      create-task.use-case.ts
      update-task-status.use-case.ts
      list-tasks.use-case.ts
      delete-task.use-case.ts
      reorder-tasks.use-case.ts
      ... (11 use cases en total)
  infrastructure/
    http/
      task.controller.ts ← Habla con Express
      task.router.ts     ← Define las rutas
      task.schemas.ts   ← Validación Zod
    persistence/
      prisma-task.repository.ts ← Implementa TaskRepository
      prisma-subtask.repository.ts ← Implementa SubtaskRepository
  tasks.module.ts       ← Registra todo en el DI container
```

Rutas de la API

```
POST  /api/auth/register
POST  /api/auth/login
POST  /api/auth/refresh

GET    /api/spaces
POST   /api/spaces
PATCH /api/spaces/:id/toggle
PATCH /api/spaces/:id/view
PATCH /api/spaces/:id/color
POST   /api/spaces/:id/template
DELETE /api/spaces/:id

GET    /api/spaces/:spaceId/tasks
POST   /api/spaces/:spaceId/tasks
PATCH /api/spaces/:spaceId/tasks/:taskId
PATCH /api/spaces/:spaceId/tasks/:taskId/status
DELETE /api/spaces/:spaceId/tasks/:taskId
POST   /api/spaces/:spaceId/tasks/reorder

GET    /api/tasks/me          ← todas las tareas del usuario autenticado

POST   /api/notifications/subscribe
DELETE /api/notifications/unsubscribe
```

9. La base de datos (Prisma + PostgreSQL)

Prisma es el ORM que conecta el código con PostgreSQL. El schema define las tablas:

```
// apps/backend/prisma/schema.prisma

model User {
  id          String   @id @default(uuid())
  email       String   @unique
  passwordHash String
  name        String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  spaces      Space[]          // un usuario tiene muchos espacios
  pushSubscriptions PushSubscription[]
}

model Space {
  id          String   @id @default(uuid())
  name        String
  viewType    SpaceViewType @default(LIST) // LIST | CALENDAR | KANBAN
  isActive    Boolean   @default(true)
  ownerId     String
  owner       User      @relation(fields: [ownerId], references: [id])

  tasks Task[]          // un espacio tiene muchas tareas
}

model Task {
  id          String   @id @default(uuid())
  title       String
  status      TaskStatus @default(TODO) // TODO | IN_PROGRESS | DONE
  priority    TaskPriority @default(MEDIUM) // LOW | MEDIUM | HIGH
  position    Int       @default(0)       // para el ordenamiento drag & drop
  dueDate     DateTime?
  spaceId     String
  space       Space     @relation(...)

  subtasks Subtask[]
}

model Subtask {
  id          String   @id @default(uuid())
  title       String
  done        Boolean @default(false)
  taskId      String
  task        Task     @relation(...)
}
```

Relaciones:

- Un **User** tiene muchos **Space**
- Un **Space** tiene muchas **Task**
- Una **Task** tiene muchos **Subtask**

Los repositorios convierten entre los objetos de Prisma y las entidades del dominio. Por ejemplo:

```
// El registro que devuelve Prisma (plano, sin lógica):
{ id: "abc", title: "Comprar leche", status: "TODO", spaceId: "xyz", ... }

// La entidad del dominio (con lógica y reglas):
Task {
  props: { id: "abc", title: "Comprar leche", status: "TODO", ... }
  changeStatus(newStatus) { ... }
  update(changes) { ... }
  toSnapshot() { ... }
}
```

10. Autenticación JWT

El sistema usa **dos tokens**:

Token	Duración	Para qué
<code>accessToken</code>	15 minutos	Autorizar peticiones normales
<code>refreshToken</code>	7 días	Obtener un nuevo accessToken sin volver a hacer login

Flujo:

1. El cliente hace login → recibe `{ accessToken, refreshToken }`
2. En cada petición protegida, envía: `Authorization: Bearer <accessToken>`
3. Cuando el `accessToken` expira, el cliente manda el `refreshToken` a `POST /api/auth/refresh` para obtener un nuevo par de tokens.

El middleware `requireAuth` verifica el token en las rutas protegidas:

```
// src/shared/infrastructure/http/auth.middleware.ts

export function requireAuth(req: AuthenticatedRequest, _res: Response, next: NextFunction): void {
  const header = req.headers.authorization;
  if (!header?.startsWith("Bearer ")) {
    throw new UnauthorizedError("Falta el encabezado de autorización");
  }

  const token = header.slice("Bearer ".length);
  const payload = jwt.verify(token, env.JWT_ACCESS_SECRET) as { sub: string };
  req.userId = payload.sub; // ← adjunta el userId al request para los controllers
  next();
}
```

11. Manejo de errores

Los errores del dominio son clases TypeScript que llevan su propio código HTTP:

```
// src/shared/domain/errors.ts

export class NotFoundError extends DomainError {
  readonly code = "NOT_FOUND";
  readonly statusCode = 404;
}

export class ValidationError extends DomainError {
  readonly code = "VALIDATION_ERROR";
  readonly statusCode = 400;
}

export class UnauthorizedError extends DomainError {
  readonly code = "UNAUTHORIZED";
  readonly statusCode = 401;
}

export class ForbiddenError extends DomainError {
  readonly code = "FORBIDDEN";
  readonly statusCode = 403;
}

export class ConflictError extends DomainError {
  readonly code = "CONFLICT";
  readonly statusCode = 409;
}
```

El middleware `errorHandler` los captura centralizadamente:

```
// Si es un error de Zod (validación del body): → HTTP 400
// Si es un DomainError: → HTTP del error (404, 401, 403, etc.)
// Si es cualquier otra cosa: → HTTP 500
```

Esto significa que en cualquier Use Case puedes hacer simplemente:

```
throw new NotFoundError("Tarea no encontrada");
```

Y Express devolverá automáticamente:

```
{ "error": { "code": "NOT_FOUND", "message": "Tarea no encontrada" } }
```

con status 404, sin necesidad de escribir `res.status(404).json(...)` en cada lugar.

12. Resumen: MVC vs Clean Architecture

Concepto	MVC	Clean Architecture (este proyecto)
Model	Clase que mapea la tabla y puede tener algo de lógica	Entidad con lógica de negocio pura. La tabla es un detalle de infraestructura.
Controller	Orquesta todo: valida, llama al modelo, decide la lógica	Solo valida el HTTP y delega. La lógica está en el Use Case.
View	Template HTML o JSON devuelto por el Controller	JSON devuelto por el Controller (el frontend es una app separada)
¿Dónde va la lógica?	En el Model o en el Controller	En el Use Case (capa de aplicación) y en las Entidades (dominio)
Dependencia de la DB	El Model conoce directamente el ORM	El Use Case solo conoce una interfaz; Prisma vive en infraestructura
Testing	Requiere mockear el ORM o levantar DB	Se puede testear el Use Case con repositorios en memoria
¿Cómo se conecta todo?	Acoplamiento directo (Controller llama al Model)	Inyección de dependencias (DI Container)

La estructura de carpetas resumida

```

src/
  server.ts          ← punto de entrada, arranca Express
  app.ts             ← registra módulos y middlewares globales
  shared/
    domain/
      errors.ts      ← errores del dominio (NotFoundError, etc.)
    infrastructure/
      di/tokens.ts   ← tokens del DI container
    http/
      auth.middleware.ts    ← verifica el JWT
      error-handler.middleware.ts ← atrapa todos los errores
      async-handler.ts     ← wrapper para errores en async/await
    persistence/
      prisma-client.ts     ← instancia singleton de Prisma
    jobs/
      notification-scheduler.ts ← scheduler de notificaciones push
    config/
      env.ts              ← variables de entorno validadas con Zod
  modules/
    auth/                 ← domain/ + application/ + infrastructure/ + auth.module.ts
    spaces/               ← domain/ + application/ + infrastructure/ + spaces.module.ts
    tasks/               ← domain/ + application/ + infrastructure/ + tasks.module.ts
    notifications/       ← domain/ + application/ + infrastructure/ + notifications.module.ts

```

Tip final: cuando tengas que agregar una feature nueva (por ejemplo, "compartir una tarea con otro usuario"), el proceso es:

1. **Dominio:** ¿hay alguna regla nueva? ¿hay que modificar una entidad o crear una?
2. **Puertos:** ¿necesitas una nueva operación de base de datos? Agrégala a la interfaz del repositorio.
3. **Use Case:** crea un nuevo archivo `share-task.use-case.ts` con la lógica.
4. **Infraestructura HTTP:** un método en el Controller y una ruta en el Router.
5. **Infraestructura Persistence:** implementa el nuevo método en el repositorio Prisma.
6. **Módulo:** registra el nuevo Use Case en el DI container.

Cada capa toca lo suyo y no más.